



Coordinación de
Educación Abierta y a Distancia
VICERRECTORADO ACADÉMICO



ADMINISTRACIÓN DE BASES DE DATOS

Actividad Autónoma 7

Nombre de la actividad: Diseño y simulación de modelos NoSQL (clave-valor, documentos, grafos y columnas).

Unidad 4: Datos no relacionales (NoSQL)

Tema 1: Introducción a NoSQL



FACULTAD DE
Ingeniería

Nombres: Obando Vivanco Josselyn Roxana

Fecha: Domingo 12 de Julio del 2026

Carrera: Ciencia de Datos e Inteligencia Artificial

Periodo académico: 2026 – S1

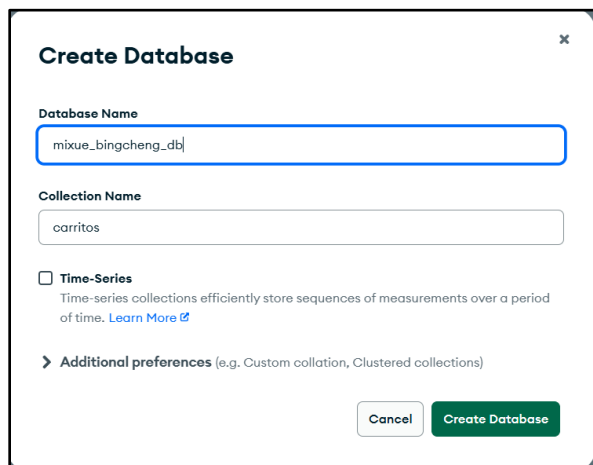
Semestre: Cuarto Semestre

Enlace del código en GitHub:

<https://github.com/RoxanaO9/Administracion-BD-NoSQL.git>

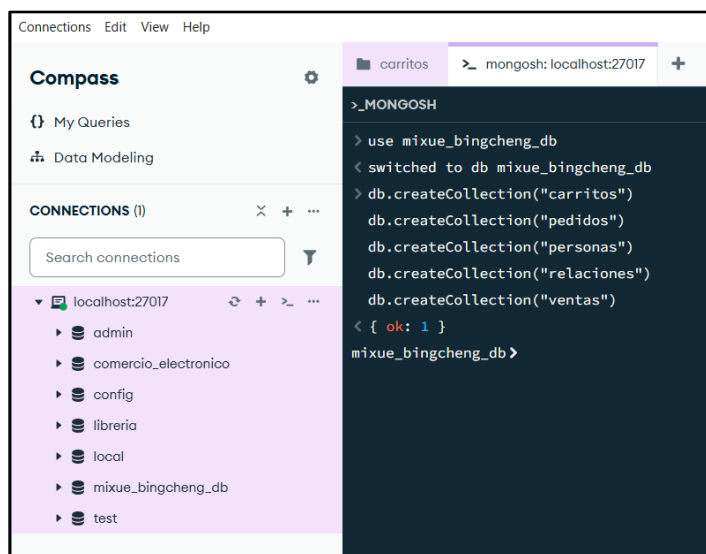
1. Creación de la base de datos

Paso 1: Creación de base de Datos



The screenshot shows the 'Create Database' dialog box in MongoDB Compass. The 'Database Name' field is filled with 'mixue_bingcheng_db'. The 'Collection Name' field is empty. There is a checkbox for 'Time-Series' which is unchecked. Below it, there is a link 'Learn More'. At the bottom, there is a 'Cancel' button and a 'Create Database' button.

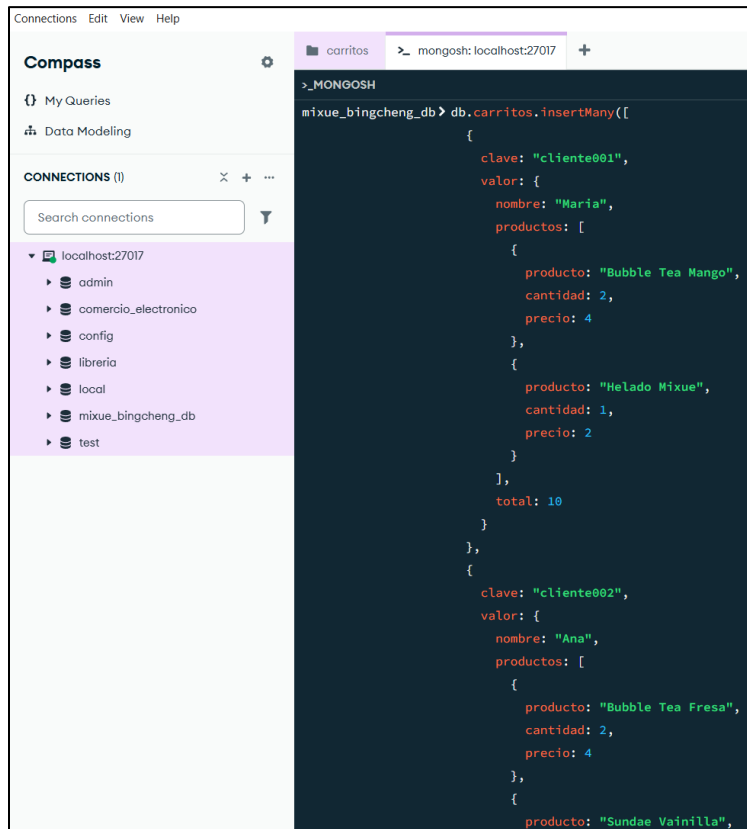
Paso 2: Creamos colecciones donde vamos a insertar los datos



2. Modelo clave valor

- Implementar un ejemplo donde se almacenen carritos de compra de clientes usando pares clave valor.
- Realizar operaciones de inserción, consulta y eliminación.

Paso 1: Implementación:



```

>_MONGOSH
mixue_bingcheng_db> db.carritos.insertMany([
  {
    clave: "cliente001",
    valor: {
      nombre: "Maria",
      productos: [
        {
          producto: "Bubble Tea Mango",
          cantidad: 2,
          precio: 4
        },
        {
          producto: "Helado Mixue",
          cantidad: 1,
          precio: 2
        }
      ]
    },
    total: 10
  },
  {
    clave: "cliente002",
    valor: {
      nombre: "Ana",
      productos: [
        {
          producto: "Bubble Tea Fresa",
          cantidad: 2,
          precio: 4
        },
        {
          producto: "Sundae Vainilla",

```

Salida:



```

< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a663c4a975e40cf53406d66'),
    '1': ObjectId('6a663c4a975e40cf53406d67'),
    '2': ObjectId('6a663c4a975e40cf53406d68'),
    '3': ObjectId('6a663c4a975e40cf53406d69'),
    '4': ObjectId('6a663c4a975e40cf53406d6a')
  }
}
mixue_bingcheng_db>

```

Paso 2: Consulta:

```
> db.carritos.find({
  clave:"cliente001"
})
< {
  _id: ObjectId('6a663c4a975e40cf53406d66'),
  clave: 'cliente001',
  valor: {
    nombre: 'Maria',
    productos: [
      {
        producto: 'Bubble Tea Mango',
        cantidad: 2,
        precio: 4
      },
      {
        producto: 'Helado Mixue',
        cantidad: 1,
        precio: 2
      }
    ],
    total: 10
  }
}
```

mixue_bingcheng_db>

Paso 3: Eliminación:

```
> db.carritos.deleteOne({
  clave:"cliente002"
})
< {
  acknowledged: true,
  deletedCount: 1
}
```

mixue_bingcheng_db>

3. Modelo orientado a documentos

- Crear una colección de pedidos que contenga documentos con subdocumentos (cliente, envío) y arreglos (items).
- Ejecutar consultas para filtrar pedidos por cliente y calcular totales.

Paso 1: Insertar datos de Pedidos:

```

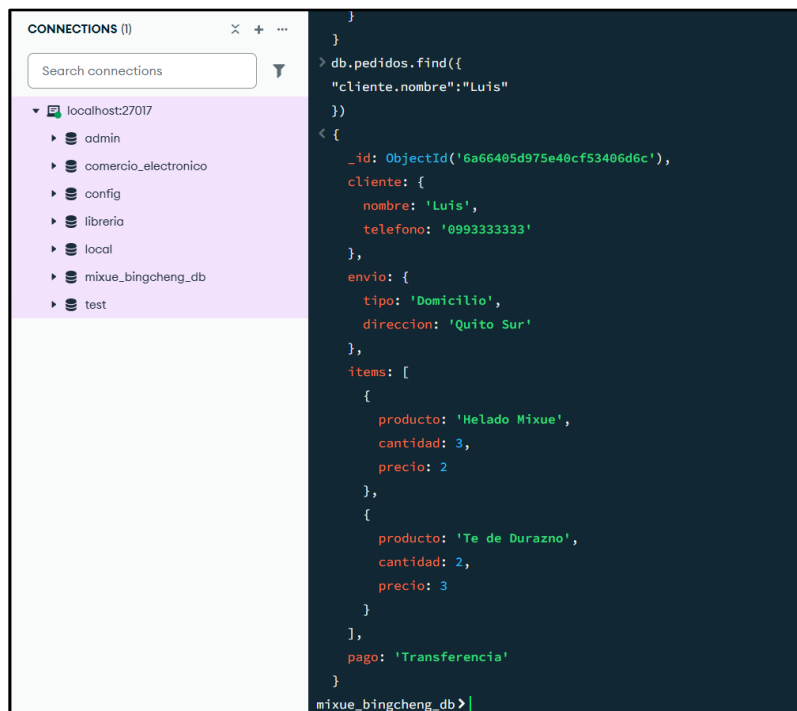
>_MONGOSH
mixue_bingcheng_db> db.pedidos.insertMany([
  {
    cliente: {
      nombre: "Maria", telefono: "0991111111"
    },
    envio: {
      tipo: "Domicilio", direccion: "Quito Norte"
    },
    items: [
      {
        producto: "Bubble Tea Mango", cantidad: 2, precio: 4
      },
      {
        producto: "Helado Mixue", cantidad: 1, precio: 2
      }
    ],
    pago: "Tarjeta"
  },
  {
    cliente: {
      nombre: "Luis", telefono: "0993333333"
    },
    envio: {
      tipo: "Domicilio", direccion: "Quito Sur"
    },
    items: [
      {
        producto: "Helado Mixue", cantidad: 3, precio: 2
      },
      {
        producto: "Te de Durazno", cantidad: 2, precio: 3
      }
    ]
  }
])
  
```

Salida:

```

< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a66405d975e40cf53406d6b'),
    '1': ObjectId('6a66405d975e40cf53406d6c'),
    '2': ObjectId('6a66405d975e40cf53406d6d'),
    '3': ObjectId('6a66405d975e40cf53406d6e')
  }
}
mixue_bingcheng_db>
  
```

Paso 2: Consultar pedidos por cliente



The screenshot shows a MongoDB CLI interface. On the left, a 'CONNECTIONS' panel lists several databases connected to localhost:27017, including 'mixue_bingcheng_db'. The main area displays a MongoDB shell prompt where a query is executed: `db.pedidos.find({'cliente.nombre': 'Luis'})`. The result is a JSON object representing a document from the 'pedidos' collection.

```

> db.pedidos.find({'cliente.nombre': 'Luis'})
< {
  _id: ObjectId('6a66405d975e40cf53406d6c'),
  cliente: {
    nombre: 'Luis',
    telefono: '0993333333'
  },
  envio: {
    tipo: 'Domicilio',
    direccion: 'Quito Sur'
  },
  items: [
    {
      producto: 'Helado Mixue',
      cantidad: 3,
      precio: 2
    },
    {
      producto: 'Te de Durazno',
      cantidad: 2,
      precio: 3
    }
  ],
  pago: 'Transferencia'
}
mixue_bingcheng_db>

```

Paso 3: Calcular totales



The screenshot shows a MongoDB CLI interface with the 'mixue_bingcheng_db' database selected. A MongoDB shell prompt displays an aggregate query using the \$project, \$sum, and \$multiply operators to calculate totals for each client.

```

mixue_bingcheng_db> db.carritos.aggregate([
  {
    $project: {
      cliente: "$valor.nombre",
      total: {
        $sum: {
          $map: {
            input: "$valor.productos",
            as: "item",
            in: {
              $multiply: [
                "$$item.cantidad",
                "$$item.precio"
              ]
            }
          }
        }
      }
    }
  }
])

```

Salida:

```
< {
  _id: ObjectId('6a663c4a975e40cf53406d66'),
  cliente: 'Maria',
  total: 10
}
{
  _id: ObjectId('6a663c4a975e40cf53406d68'),
  cliente: 'Luis',
  total: 12
}
{
  _id: ObjectId('6a663c4a975e40cf53406d69'),
  cliente: 'Sofia',
  total: 12
}
{
  _id: ObjectId('6a663c4a975e40cf53406d6a'),
  cliente: 'Javier',
  total: 16
}
mixue_bingcheng_db> |
```

4. Modelo de grafos

- Simular una red universitaria que relacione estudiantes y profesores.
- Insertar nodos (estudiantes y docentes) y aristas (amistad, colaboración, materias dictadas).
- Realizar consultas para identificar conexiones.

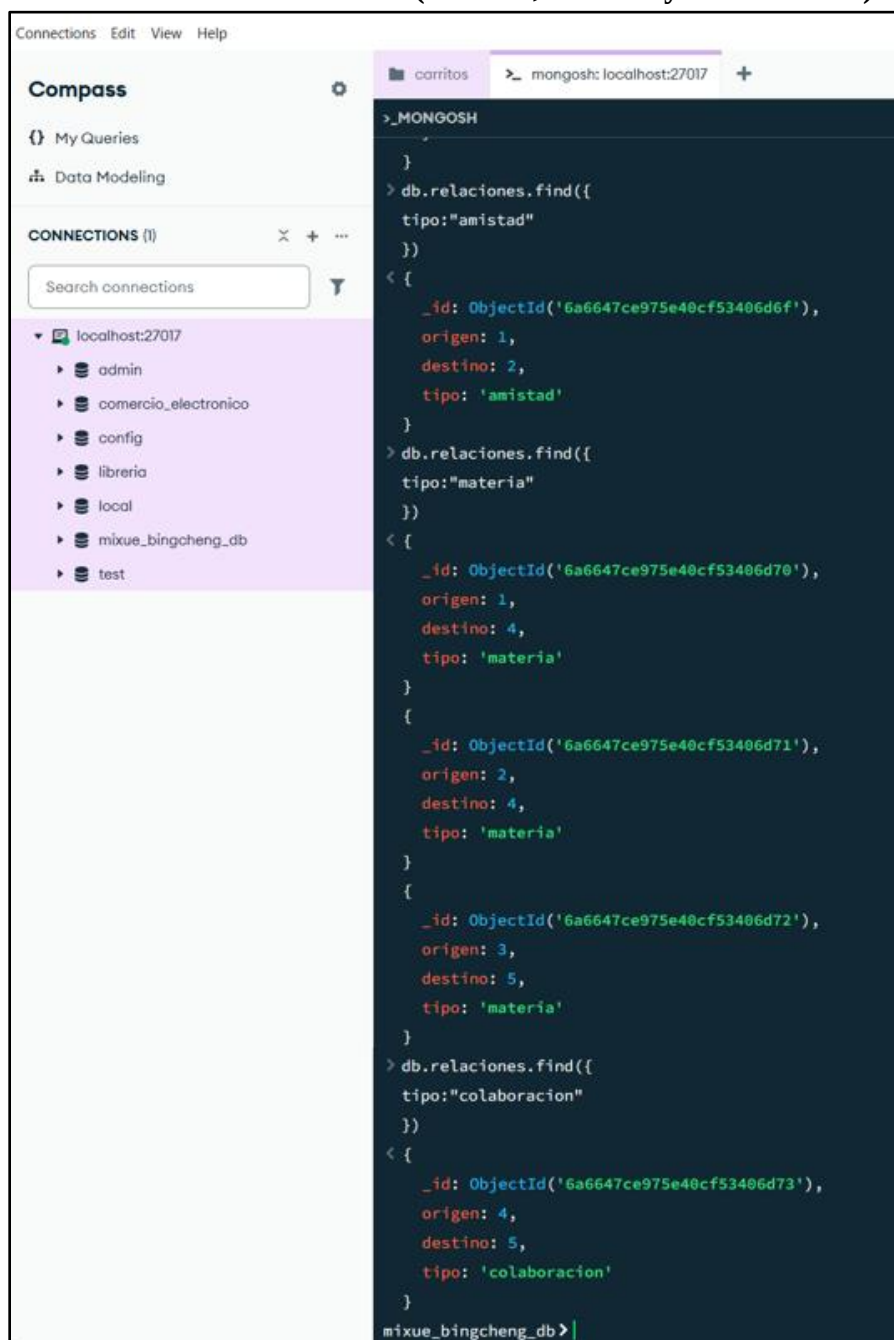
Paso 1: Insertar nodos (Personas)

```
> db.personas.insertMany([
  { _id: 1, nombre: "Maria", tipo: "Estudiante" },
  { _id: 2, nombre: "Luis", tipo: "Estudiante" },
  { _id: 3, nombre: "Sofia", tipo: "Estudiante" },
  { _id: 4, nombre: "Profesor Carlos", tipo: "Docente" },
  { _id: 5, nombre: "Profesora Andrea", tipo: "Docente" }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': 1,
    '1': 2,
    '2': 3,
    '3': 4,
    '4': 5
  }
}
mixue_bingcheng_db>
```

Paso 2: Insertar aristas

```
> db.relaciones.insertMany([
  { origen: 1, destino: 2, tipo: "amistad" },
  { origen: 1, destino: 4, tipo: "materia" },
  { origen: 2, destino: 4, tipo: "materia" },
  { origen: 3, destino: 5, tipo: "materia" },
  { origen: 4, destino: 5, tipo: "colaboracion" }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a665a85ef37af748866d456'),
    '1': ObjectId('6a665a85ef37af748866d457'),
    '2': ObjectId('6a665a85ef37af748866d458'),
    '3': ObjectId('6a665a85ef37af748866d459'),
    '4': ObjectId('6a665a85ef37af748866d45a')
  }
}
mixue_bingcheng_db>
```


Paso 3: Consultar conexiones (amistad, materia y colaboracion)



Paso 3.1: Consultas de grafos (Red de María y Nombres completos)

- **Consulta Red de María**

```
> db.relaciones.find({
  $or: [{ origen: 1 }, { destino: 1 }]
})
< {
  _id: ObjectId('6a665a85ef37af748866d456'),
  origen: 1,
  destino: 2,
  tipo: 'amistad'
}
{
  _id: ObjectId('6a665a85ef37af748866d457'),
  origen: 1,
  destino: 4,
  tipo: 'materia'
}
mixue_bingcheng_db>
```

- **Consulta para ver nombre completo**

```
mixue_bingcheng_db> db.relaciones.aggregate([
  {
    $lookup: {
      from: "personas",
      localField: "origen",
      foreignField: "_id",
      as: "origen_info"
    }
  },
  {
    $lookup: {
      from: "personas",
      localField: "destino",
      foreignField: "_id",
      as: "destino_info"
    }
  },
  {
    $project: {
      "origen_nombre": { $arrayElemAt: ["$origen_info.nombre", 0] },
      "destino_nombre": { $arrayElemAt: ["$destino_info.nombre", 0] },
      "tipo": 1,
      "_id": 0
    }
  }
])
```

Salida:

```
< {
  tipo: 'amistad',
  origen_nombre: 'Maria',
  destino_nombre: 'Luis'
}
{
  tipo: 'materia',
  origen_nombre: 'Maria',
  destino_nombre: 'Profesor Carlos'
}
{
  tipo: 'materia',
  origen_nombre: 'Luis',
  destino_nombre: 'Profesor Carlos'
}
{
  tipo: 'materia',
  origen_nombre: 'Sofia',
  destino_nombre: 'Profesora Andrea'
}
{
  tipo: 'colaboracion',
  origen_nombre: 'Profesor Carlos',
  destino_nombre: 'Profesora Andrea'
}
mixue_bingcheng_db>|
```

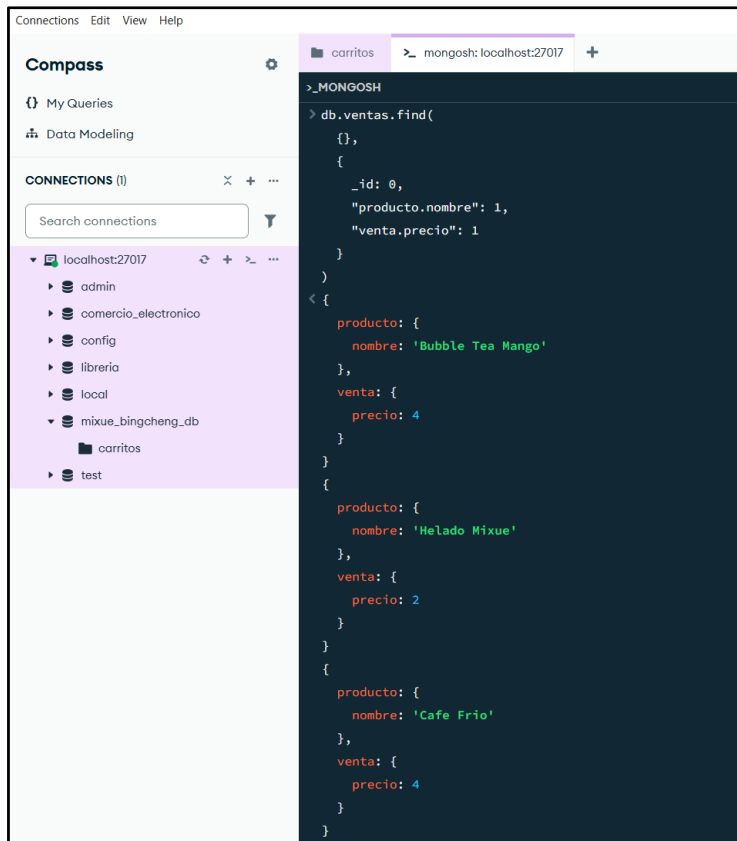
5. Modelo de columnas

Paso 1: Inserción de registros con familias de columnas

```
> db.ventas.insertMany([
  {
    cliente: { nombre: "Maria" },
    producto: { nombre: "Bubble Tea Mango" },
    venta: { cantidad: 2, precio: 4 }
  },
  {
    cliente: { nombre: "Luis" },
    producto: { nombre: "Helado Mixue" },
    venta: { cantidad: 3, precio: 2 }
  },
  {
    cliente: { nombre: "Javier" },
    producto: { nombre: "Cafe Frio" },
    venta: { cantidad: 2, precio: 4 }
  }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a665d14ef37af748866d45b'),
    '1': ObjectId('6a665d14ef37af748866d45c'),
    '2': ObjectId('6a665d14ef37af748866d45d')
  }
}
mixue_bingcheng_db>
```

Paso 2: Consultas

Paso 2.1: Consulta 1: Columnas específicas (producto y precio)



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel lists the database 'mixue_bingcheng_db' and the collection 'carritos'. The main editor on the right shows a query in the MongoDB shell:

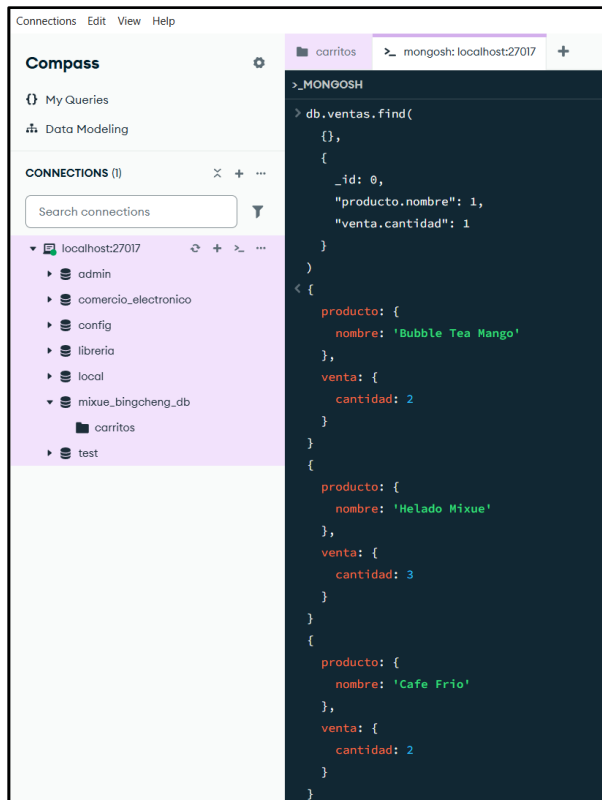
```
> db.ventas.find(
  {},
  {
    _id: 0,
    "producto.nombre": 1,
    "venta.precio": 1
  }
)
< {
  producto: {
    nombre: 'Bubble Tea Mango'
  },
  venta: {
    precio: 4
  }
}
{
  producto: {
    nombre: 'Helado Mixue'
  },
  venta: {
    precio: 2
  }
}
{
  producto: {
    nombre: 'Cafe Frio'
  },
  venta: {
    precio: 4
  }
}
```

The query filters for all documents and projects only the product name and the price, resulting in three documents as shown in the output.

Paso 2.2: Consulta 2: Ingresos por producto (ya la tienes)

```
> db.ventas.aggregate([
  {
    $group: {
      _id: "$producto.nombre",
      ingresos: {
        $sum: {
          $multiply: ["$venta.cantidad", "$venta.precio"]
        }
      }
    }
  }
])
< {
  _id: 'Cafe Frio',
  ingresos: 8
}
{
  _id: 'Bubble Tea Mango',
  ingresos: 8
}
{
  _id: 'Helado Mixue',
  ingresos: 6
}
mixue_bingcheng_db>
```

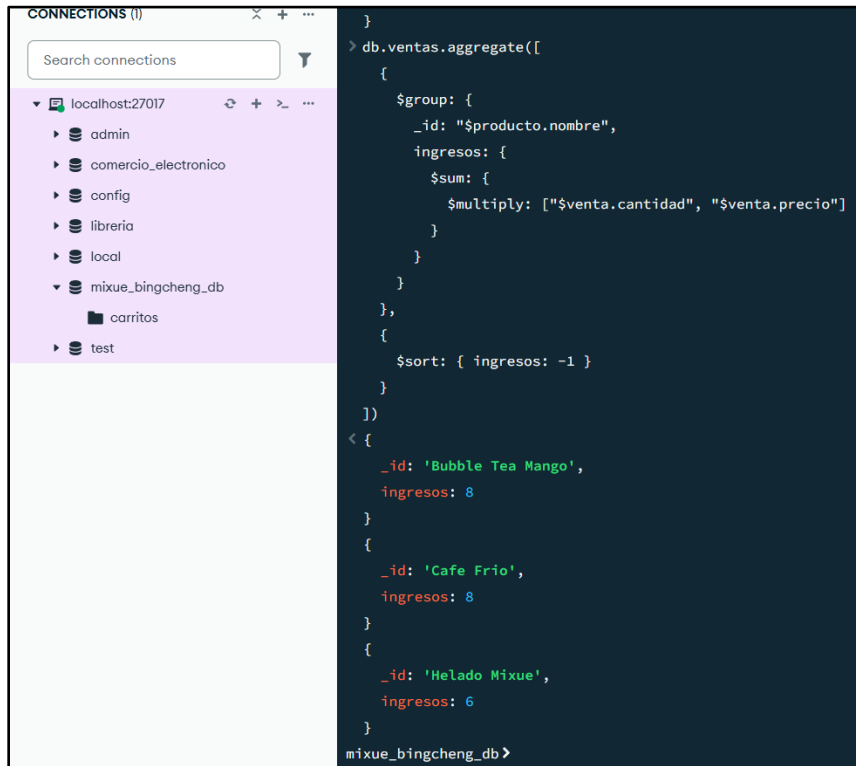
Paso 2.3: Consulta 3: Solo productos y cantidades



The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel lists the database 'mixue_bingcheng_db' with a collection named 'carritos'. The main editor on the right shows a query in the 'MONGOSH' tab:

```
> db.ventas.find(
  {
    _id: 0,
    "producto.nombre": 1,
    "venta.cantidad": 1
  }
)
< {
  producto: {
    nombre: 'Bubble Tea Mango'
  },
  venta: {
    cantidad: 2
  }
}
{
  producto: {
    nombre: 'Helado Mixue'
  },
  venta: {
    cantidad: 3
  }
}
{
  producto: {
    nombre: 'Cafe Frio'
  },
  venta: {
    cantidad: 2
  }
}
```

Paso 2.4: Consulta 4: Ingresos por producto ordenados

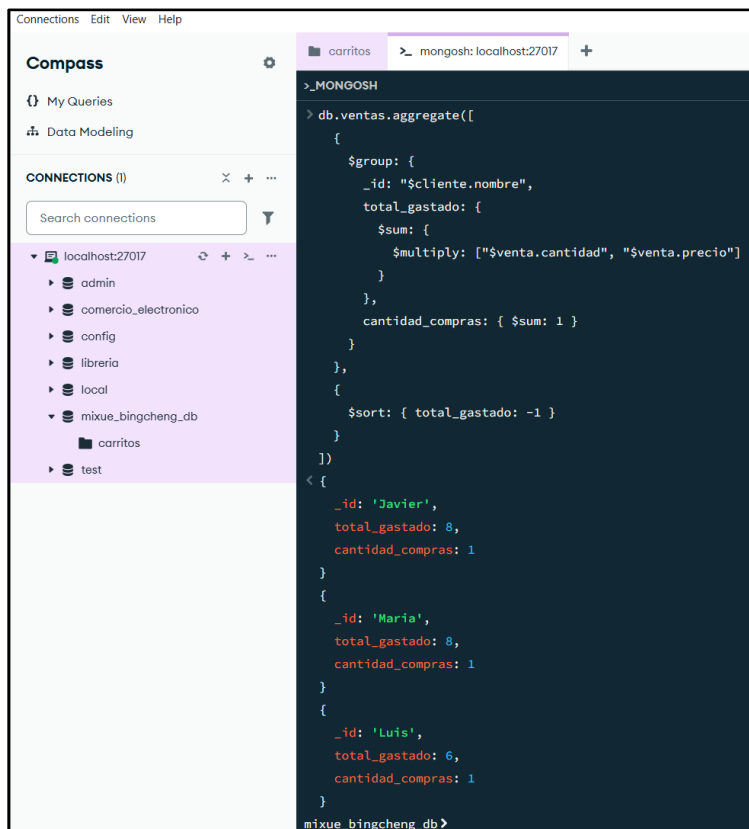


The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS (1)' panel lists the connection 'localhost:27017' and the database 'mixue_bingcheng_db'. The main editor displays a MongoDB aggregation query:

```

> db.ventas.aggregate([
  {
    $group: {
      _id: "$producto.nombre",
      ingresos: {
        $sum: {
          $multiply: ["$venta.cantidad", "$venta.precio"]
        }
      }
    }
  },
  {
    $sort: { ingresos: -1 }
  }
])
< {
  _id: 'Bubble Tea Mango',
  ingresos: 8
}
{
  _id: 'Cafe Frio',
  ingresos: 8
}
{
  _id: 'Helado Mixue',
  ingresos: 6
}
mixue_bingcheng_db>
  
```

Paso 2.5: Consulta 5: Gasto total por cliente



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS (1)' panel lists the connection 'localhost:27017' and the database 'mixue_bingcheng_db'. The main editor displays a MongoDB aggregation query:

```

>_MONGOSH
> db.ventas.aggregate([
  {
    $group: {
      _id: "$cliente.nombre",
      total_gastado: {
        $sum: {
          $multiply: ["$venta.cantidad", "$venta.precio"]
        }
      },
      cantidad_compras: { $sum: 1 }
    }
  },
  {
    $sort: { total_gastado: -1 }
  }
])
< {
  _id: 'Javier',
  total_gastado: 8,
  cantidad_compras: 1
}
{
  _id: 'Maria',
  total_gastado: 8,
  cantidad_compras: 1
}
{
  _id: 'Luis',
  total_gastado: 6,
  cantidad_compras: 1
}
mixue_bingcheng_db>
  
```

Paso 2.6: Consulta 6: Promedio de ingreso por producto

```
> db.ventas.aggregate([
  {
    $group: {
      _id: "$producto.nombre",
      promedio_ingreso: {
        $avg: {
          $multiply: ["$venta.cantidad", "$venta.precio"]
        }
      }
    }
  }
])
< {
  _id: 'Helado Mixue',
  promedio_ingreso: 6
}
{
  _id: 'Bubble Tea Mango',
  promedio_ingreso: 8
}
{
  _id: 'Cafe Frio',
  promedio_ingreso: 8
}
mixue_bingcheng_db >
```

6. Comparación y análisis:

Comparación de modelos:

<i>Modelo</i>	Ventajas frente al modelo relacional
<i>Clave-Valor</i>	Este modelo ofrece un acceso mucho más rápido a la información cuando se busca mediante una clave única. Además, su estructura es sencilla y puede escalar fácilmente para soportar una gran cantidad de consultas.
<i>Documentos</i>	Permite almacenar datos con estructuras flexibles, por lo que no es necesario modificar toda la base de datos cuando cambia la información. También reduce la necesidad de realizar uniones entre tablas, haciendo las consultas más simples.
<i>Grafos</i>	Está diseñado para gestionar relaciones entre datos de forma más eficiente que el modelo relacional. Esto facilita encontrar conexiones entre diferentes elementos sin recurrir a consultas complejas.
<i>Columnas</i>	Organiza la información de manera que las consultas analíticas sean más rápidas. Además, optimiza el almacenamiento y ofrece un mejor rendimiento cuando se trabaja con grandes volúmenes de datos.

Análisis de escenarios de uso

Cada modelo de base de datos es más útil dependiendo de las necesidades del sistema.

- **Modelo Clave-Valor:** Es ideal cuando se necesita acceder a los datos de forma casi inmediata. Se usa mucho para guardar sesiones de usuarios, carritos de compra o información temporal. En Mixue Bingcheng, serviría para almacenar los productos que un cliente va agregando a su carrito y actualizarlos al instante.
- **Modelo Orientado a Documentos:** Funciona muy bien cuando la información puede variar de un registro a otro. En Mixue, se podría usar para guardar los pedidos completos en un solo documento, incluyendo los datos del cliente, la dirección de entrega y los productos comprados, lo que hace más sencilla la administración de los pedidos.
- **Modelo de Grafos:** Es la mejor opción cuando lo más importante son las relaciones entre los datos. Por ejemplo, en una universidad permitiría mostrar fácilmente las conexiones entre estudiantes, profesores, materias o proyectos, haciendo más simple consultar cómo se relacionan entre sí.
- **Modelo de Columnas:** Se recomienda para sistemas que manejan enormes cantidades de datos y necesitan generar análisis o reportes de manera constante. En Mixue Bingcheng, sería útil para revisar las ventas, identificar los productos más populares y comparar el desempeño de las diferentes sucursales de forma rápida.